

Fórmulas de Daño y Escudo

Spec v3 — modelo N×M. Derivada del código, no del diseño en papel.

VERSIÓN	v3 — decisión de diseño del 2026-08-09. Supersede v2 (donde <code>dmg_base_PJ</code> , <code>bonos_PJ</code> y <code>bono_combo</code> quedaban fuera del producto).
RAMA	<code>Feature#0046_Chest</code>
COMMIT BASE	<code>fc0f08cb</code> — test(localization): allow Buenardo as identical-by-design tier name
FECHA	2026-08-11
FUENTE DE VERDAD	Assets/Scripts/Rollgeon/Combat/Damage/PlayerComboDamage.cs
ALCANCE	Ataque y escudo de combo del jugador , y el pipeline de aplicación que atraviesa todo el daño del juego (jugador y enemigos).

Índice

1.	La fórmula canónica	N × M
2.	Anatomía de N — los términos aditivos	
3.	Anatomía de M — los términos multiplicativos	
4.	El redondeo	RoundNxM
5.	Daño: de dónde sale <code>comboBase</code>	
6.	Escudo: de dónde sale <code>shieldBase</code> y el gate	
7.	Divergencias daño vs. escudo	
8.	El caso sin combo (fallback del dado más alto)	
9.	El pipeline de aplicación del daño	
10.	El escudo como recurso: aplicación, absorción y reset	
11.	Daño de enemigos y otras fuentes	
12.	Preview del HUD vs. golpe real	
13.	Ejemplos numéricos trabajados	7 casos
14.	Datos vigentes del Warrior	
15.	Gotchas, deudas y hallazgos de la revisión	
16.	Apéndice A — mapa de archivos	
17.	Apéndice B — el log de diagnóstico en runtime	

1 La fórmula canónica

Daño de combo y escudo de combo del jugador se resuelven con **una sola función**. No hay dos fórmulas: hay una fórmula y dos bases distintas.

`PlayerComboShield.Resolve` delega en `PlayerComboDamage.Resolve` pasándole `PlayerComboFormulaKind.Shield`. Ese enum **solo cambia el rótulo del log**; la aritmética es idéntica byte por byte.

FÓRMULA V3 — PLAYERCOMBODAMAGE.CS:114-116

$$N = \text{comboBase} + \text{Attack.Value} + (\text{Attack.ModifiedValue} - \text{Attack.Value}) + \Sigma(\text{caras contribuyentes}) + \Sigma(\text{BonusComboDamage de los 3 canales})$$
$$M = (\prod \text{ComboDamageMultiplier de los 3 canales}) \times \text{abilityMultiplier}$$
$$\text{RESULTADO} = \max(0, \text{round}(N \times M, \text{AwayFromZero}))$$

Con un cortocircuito por encima de todo:

GATE DE BLOQUEO

si **CUALQUIERA** de los 3 canales levantó **BlockComboDamage**
→ RESULTADO = 0 (el producto ni se calcula)

1.1 Qué cambió respecto de v2

En la v2, `dmg_base_PJ`, `bonos_PJ` y `bono_combo` quedaban *fuera* del producto: se sumaban después de multiplicar. En v3 **todo lo aditivo vive en N y todo N se escala por M**. Un multiplicador de $\times 1.5$ ahora también escala tu ATQ base.

El otro cambio: **desapareció el multiplicador por tipo de dado** (el viejo `EV/3.5`). Un d20 pesa más que un d6 porque sus caras son más altas y entran a Σ caras, no porque tenga un coeficiente propio.

PUREZA

`Resolve` es una lectura pura: llamarla N veces devuelve el mismo resultado sin side-effects. Por eso el preview del HUD puede usar exactamente la misma función que el golpe real, en vez de re-derivar la fórmula en la UI (que era la causa del desfase reportado en su momento).

1.2 La firma real

```
int Resolve(
    Guid                sourceId,           // dueño del stat Attack
    int                 comboBaseDamage,    // base de combo (daño) o base de escudo
    IReadOnlyList<ContributingDie> contributingDice, // slot + cara + tipo
    float               abilityMultiplier,  // perilla por habilidad
    PlayerComboFormulaKind kind,           // Damage | Shield (solo rótulo)
    out DamageBreakdown breakdown)         // desglose para UI/logging
```

El `out DamageBreakdown` expone cada término por separado con la invariante `Final == RoundNxM(N, M)` (o `0` si `Blocked`). La UI de breakdown y el logger consumen esa struct — **nunca recalculan**.

2 Anatomía de N — los términos aditivos

TÉRMINO	CAMPO EN DAMAGEBREAKDOWN	ORIGEN EXACTO
comboBase	ComboBase	DAÑO ComboDetectionResult.BaseDamage ESCUDO ContractSheet.GetShieldBase(comboId) Ver §5 y §6.
dmg_base_PJ	AttackBase	AttributesManager.GetAttribute<Attack>(sourceId).Value — el valor crudo, sin modificadores. Se siembra desde ClassHeroS0.BaseAttack en RunController.
bonos_PJ	AttackBonus	Attack.ModifiedValue - Attack.Value. ModifiedValue aplica los modificadores Intrinsic en orden de inserción; los direccionales (Outgoing / Incoming) no entran acá.
Σ caras	FacesSum	Suma de ContributingDie.Face de los dados que formaron el combo. Ver §2.1.
bono_combo	AdditiveBonus	Suma de BonusComboDamage de los tres canales de scratch. Ver §3.1.
N		N

DEGRADADO SILENCIOSO

Si el ServiceLocator no tiene AttributesManager registrado, o la entidad no tiene atributo Attack, **ambos términos valen 0** sin warning. La fórmula sigue corriendo con el resto.

2.1 Cómo se resuelven las caras contribuyentes

Un ContributingDie viaja como tripla indivisible — **slot del bag + cara tirada + tipo** — para que las tres piezas no puedan desalinearse entre el payload de preview y el contexto de impacto.

```
readonly struct ContributingDie {  
    int    BagSlot;    // para que la UI apunte al dado físico en pantalla; -1 si no aplica  
    int    Face;       // término de Σcaras  
    DiceType Type;    // arte/color - NO entra a la fórmula  
}
```

El mapeo lo hace ContributingDiceResolver.ResolveFromContext, en tres saltos:

- ComboDetectionResult.ContributingIndices viene en el espacio de índices del **subset holdeado**, no del bag.
- EffectContext.KeptDiceOriginalIndices traduce subset → slot del bag. Si es null, se asume identidad (los índices ya son slots).
- Las caras salen de KeptDice ?? DiceResult; el tipo, de IDiceEnchantmentService.Bag.Dice[bagSlot].

Si el enchantment service no está registrado o no hay bag, el resolver devuelve null y **Σcaras queda en 0**. Esto importa para el fallback sin combo (§8), que tiene un camino alternativo justamente por eso.

SOLO LAS CARAS DEL COMBO

Un dado holdeado que *no* formó parte del combo ganador no aporta nada. Un Par sobre [6, 6, 5, 4, 1] suma 12, no 22: el 5, el 4 y el 1 quedan afuera aunque estén en la mesa.

3 Anatomía de M — los términos multiplicativos

TÉRMINO	CAMPO	ORIGEN
<code>scratch_multiplier</code>	<code>ScratchMultiplier</code>	Producto de <code>ComboDamageMultiplier</code> de los 3 canales. Arranca en <code>1f</code> .
<code>ability_multiplier</code>	<code>AbilityMultiplier</code>	El campo <code>_comboMultiplier</code> del <code>EffDealDamage</code> / <code>EffAddShield</code> de esa habilidad. Perilla de balance por acción (ej. un "golpe rápido" a 0.75). Mínimo autorable: <code>0.01</code> .
M		M

3.1 Los tres canales de scratch

Un *scratch* es un acumulador efímero donde las pasivas, encantamientos e ítems escriben sus contribuciones durante la ventana de dispatch. La fórmula lee los tres, **en este orden**:

#	CANAL	SERVICIO / PROPIEDAD	CUÁNDO SE PUEBLA
1	Pasivas de combo	<code>IComboPassiveService.LastComboScratch</code>	<i>at-match</i> — al detectarse el combo
2	Encantamientos de dados	<code>IDiceEnchantmentService.LastComboScratch</code>	<i>at-match</i>
3	Ítems / combo jugado	<code>IComboPlayService.LastPlayScratch</code>	<i>at-played</i> — al confirmar la jugada

Los tres exponen la misma struct (`EnchantmentScratch`) y los tres campos relevantes se componen así:

CAMPO	COMPOSICIÓN ENTRE CANALES	EFECTO
<code>BonusComboDamage</code>	suma (<code>+=</code>)	entra a N
<code>ComboDamageMultiplier</code>	multiplica (<code>*=</code>)	entra a M
<code>BlockComboDamage</code>	OR lógico (<code> =</code>)	anula el resultado

Como la composición es conmutativa en los tres casos, **el orden de dispatch dentro de un canal no altera el resultado**. `EffMultiplyComboDamage` hace `Scratch.ComboDamageMultiplier *= _multiplier`; dos efectos de $\times 1.5$ dan $\times 2.25$, no $\times 3$.

3.2 Por qué el canal *at-played* es distinto

Los canales 1 y 2 leen `LastComboScratch`. El canal 3 lee `LastPlayScratch`, que **persiste más allá de la ventana de jugada**. La razón es de timing, no de diseño:

El daño real está diferido al **frame de impacto** de la animación (el `EffDealDamage` vive dentro de un `EffPlaySequence` que arranca en el evento `hit`). Para cuando se ejecuta, la ventana de combo jugado ya está cerrada. Si el bono viviera en el scratch efímero, se habría perdido.

Se limpia al inicio del turno, de modo que el preview no arrastre bonos jugados de un turno anterior.

ESCUDO TAMBIÉN

Los tres canales aplican **igual al escudo**. Un `BonusComboDamage` suma escudo en la fase de defensa, y `BlockComboDamage` también bloquea escudo — no solo daño.

3.3 Journal de atribución

Cada canal expone un `Journal` de `ScratchContribution` (qué fuente aportó cuánto). `Resolve` los concatena en `DamageBreakdown.Sources` en el orden de agregación — pasivas → encantos → at-played. Es un journal agregado por fuente, **no un timeline global**. Vale `null` si nadie aportó (así no aloca en el caso común).

4 El redondeo RoundNxM

El producto `N × M` es un `float`; el daño es un `int`. El puente es una función pública y explícita:

```
public static int RoundNxM(int n, float m)
    => Math.Max(0, (int)Math.Round(n * (double)m, MidpointRounding.AwayFromZero));
```

Tres decisiones medidas en una línea:

1. `MidpointRounding.AwayFromZero` — las mitades siempre suben: `6.5 → 7`, `42.5 → 43`. `Mathf.RoundToInt` haría *banker's rounding* (mitad al par más cercano): `6.5 → 6`, `42.5 → 42`. Es la diferencia que la función existe para evitar.
2. **Promoción a `double`** antes de multiplicar — evita que el error de representación de un `float` como `0.1f` empuje un valor justo por debajo del midpoint.
3. `Math.Max(0, ...)` — el resultado nunca es negativo, aunque un multiplicador negativo mal autorado lo llevara ahí.

Es `public` a propósito: el preview del HUD la llama directamente para reproducir **exactamente** el número del golpe real cuando tiene que re-redondear con un bono de ítem encima (§12).

INCONSISTENCIA DETECTADA

Esta garantía **se rompe en el pipeline**. La etapa de weakness usa `Mathf.RoundToInt(damage * weakMult)` — *banker's rounding* — en vez de `RoundNxM`. Ver §15.1 para el caso concreto donde difieren.

5 Daño: de dónde sale comboBase

Para el daño, `comboBase` es el `BaseDamage` del `ComboDetectionResult`. Su cálculo tiene tres capas.

5.1 Capa 1 — el base plano del SO

Cada combo es un `BaseComboSO` con un campo `_baseDamage` (rango 0..500, editable sin recompilar). Valores vigentes:

COMBO	COMBOID	BASEDAMAGE	PRIORITY	REGLA
Higher Number	<code>combo.higher_number</code>	5	5	fallback: cualquier selección no vacía
Par	<code>combo.pair</code>	8	8	2 dados iguales
Doble Par	<code>combo.double_pair</code>	15	15	2 pares
Trío	<code>combo.trio</code>	22	22	3 iguales
Fuerza Bruta	<code>combo.brute_force</code>	30 *	30	los 5 dados en mitad superior de su rango
Full House	<code>combo.full_house</code>	35	35	trío + par
Escalera	<code>combo.ladder</code>	40	40	5 consecutivos
Póker	<code>combo.poker</code>	55	55	4 iguales
Generala	<code>combo.generala</code>	90	<code>int.MaxValue</code>	5 iguales

* **Fuerza Bruta no usa este número para el daño.** Ver §5.3 y §15.3.

`Priority` por defecto es `_baseDamage`: cuando varios combos matchean la misma tirada, gana el de mayor prioridad. Generala overridea a `int.MaxValue` como *hard rule*, para que ningún ajuste de balance de Póker pueda destronarla.

5.2 Capa 2 — el override por clase

`ContractSheet.BaseDamageTable` permite que una clase redefina el base plano de un combo sin tocar el SO global:

```
int? GetBaseDamageOverride(string comboId) // scan lineal, null si no hay entrada
int GetBaseDamage(BaseComboSO combo)
=> GetBaseDamageOverride(combo.ComboId) ?? combo.BaseDamage;
```

EL OVERRIDE NO MUEVE LA PRIORIDAD

`Priority` sigue leyendo el `BaseDamage` global del SO. Una clase puede pegar el doble con Par sin que Par le gane a Trío en la selección. La tabla cambia **cuánto pega**, no **qué combo se elige**.

5.3 Capa 3 — los combos de base dinámica

Tres combos overridean `Detect` para devolver un `BaseDamage` calculado. En los tres, el `flatBaseOverride` de la tabla por clase reemplaza **solo el piso plano**; la parte variable suma encima.

COMBO	FÓRMULA DE <code>BASEDAMAGE</code>	<code>CONTRIBUTINGINDICES</code>
Higher Number	<code>(override ?? 5) + valor del dado más alto</code>	solo el índice del dado más alto (primera ocurrencia ante empate)
Suma X	<code>(override ?? 25) + X × hits</code> hits = cantidad de dados que muestran X	los índices de los dados que muestran X
Fuerza Bruta	<code>(override ?? 5) + Σ(los 5 valores)</code>	los 5 índices

DOBLE CONTEO ACEPTADO

En estos tres combos la cara del dado entra **dos veces** a N: una dentro de `comboBase` y otra por Σ caras. Está documentado como deuda de balance conocida en `PlayerComboDamage.cs:41-42` y aceptado hasta el próximo pase. Ver el ejemplo trabajado #3 (§13) para el impacto numérico.

Notas por combo

- **Fuerza Bruta** es el único combo cuya regla depende del `DiceType`, no solo del valor: "mitad superior" es `valor > MaxFace/2` (d6 → {4,5,6}, d8 → {5..8}, d12 → {7..12}). Si el call site no provee los tipos, asume d6. Además exige los **5 dados del bag**, no un subset holdeado.
- **Suma X** no tiene ningún `.asset` autorado en el proyecto: la clase existe y está testeada, pero no hay instancia en `ComboCatalog`.
- **Higher Number** usa `valor absoluto`, no valor relativo al rango del dado. Un d4 en 4 pierde contra un d20 en 5.

6 Escudo: de dónde sale `shieldBase` y el gate

El escudo **nunca** lee `combo.BaseDamage`. Esa fue la causa raíz de BUG-021 (escudo trivial): la tabla de ataque se colaba en la fórmula defensiva. Hoy la base sale de una tabla propia, fija e independiente.

```
int GetShieldBase(string comboId) // ContractSheet.ShieldBaseTable, scan lineal
// sin entrada ⇒ 0. NO hay fallback a un default global.
```

6.1 El gate de base 0

Es la única divergencia aritmética real entre daño y escudo:

```
PLAYERCOMBOSHIELD.CS:33-38
```

```
si shieldBase <= 0
  → retorna 0 y la fórmula ni se ejecuta
  → breakdown = struct vacía con Kind = Shield
```

Por qué existe. La `ShieldBaseTable` es *opt-in por clase*: la ausencia de entrada significa "esta clase no genera escudo con ese combo". Sin el gate, la fórmula correría con `comboBase = 0` y todo combo daría escudo $\approx \text{Attack.ModifiedValue} + \Sigma \text{caras}$ — que es exactamente lo que no se quiere.

La división de responsabilidades es limpia: **el gate decide si hay escudo; la fórmula decide cuánto.**

6.2 El segundo gate, en el efecto

`EffAddShield.ResolveComboShield` agrega dos condiciones previas, antes incluso de llegar a `PlayerComboShield`:

```
var sheet = IPlayerService?.CurrentHero?.Sheet;
if (sheet == null || string.IsNullOrEmpty(combo.ComboId)) return 0;
```

Un `ComboId` vacío es el resultado sintético de un *action roll* (Heal, Forzar Puerta): significa "sin datos por combo" y no genera escudo.

6.3 Sin cap

La v3 eliminó el cap de escudo. El freno anti-inmunidad son dos cosas del sistema, no un número en la fórmula:

- El **reset por turno** (§10.3): el escudo vuelve a 0 al empezar tu propio turno.
- La **escala ×10 del daño enemigo** (vida de PJ = 100).

7 Divergencias daño vs. escudo

Tabla completa de lo que difiere. Todo lo que no está acá es idéntico.

ASPECTO	DAÑO	ESCUDO
Aritmética	idéntica — misma función, mismo redondeo, mismos canales	
Base del combo	<code>ComboDetectionResult.BaseDamage</code> (SO global + override por clase + parte dinámica)	<code>ContractSheet.GetShieldBase(comboId)</code> tabla fija, independiente, opt-in
Base ausente	no aplica — siempre hay base (puede ser 0 y la fórmula corre igual)	gate: retorna 0 sin ejecutar la fórmula
Comboid vacío	cae al fallback del dado más alto (§8)	retorna 0
Post-fórmula	entra al <code>DamagePipeline</code> como <code>BaseDamage</code> (weakness, escudo del target, HP)	no pasa por el pipeline: se suma directo al stat <code>Shield</code>
Efecto	<code>EffDealDamage</code>	<code>EffAddShield</code>
Target	ocupantes de las celdas seleccionadas; fallback a <code>TargetGuid</code> . Puede ser multi-target.	ocupante de la primera celda seleccionada; fallback al propio <code>SourceGuid</code> . Single target.
Floating number	<code>BehaviorValueKey.FloatingDamage</code>	<code>BehaviorValueKey.FloatingShield</code>

CONSECUENCIA PRÁCTICA

Como el escudo no atraviesa el pipeline, **ninguna debilidad, ninguna mitigación y ningún clamp de HP mínimo lo tocan**. Un enemigo con debilidad a Trío no hace que tu Trío defensivo genere ×1.5 escudo.

8 El caso sin combo (fallback del dado más alto)

Cuando la tirada no matchea ningún combo, el ataque **no es 0**. El GD §5 define un daño mínimo — "el número más alto" — y la v3 lo hace entrar a la misma fórmula.

8.1 Qué dado se elige

`EffDealDamage.ResolveNoComboFallback` prioriza `KeptDice` sobre `DiceResult`. Es una decisión de diseño explícita: **cuentan los dados que el jugador eligió usar**, no la tirada completa. Si holdeaste solo un 3 y quedó un 6 sin holdear en la mesa, el mínimo es 3.

8.2 Los dos caminos, mismo N

La cara del dado más alto entra a N **una sola vez**. Por cuál término entra depende de si hay detalle de bag disponible:

SITUACIÓN	COMBOBASE	ΣCARAS	RESULTADO
Hay detalle de bag (<code>enchantment service</code> registrado)	0	la cara más alta	conserva la atribución del dado para la UI
No hay detalle de bag	la cara más alta	0 (dice es null)	mismo N, sin atribución visual

```
// EffDealDamage.cs:154-156
return PlayerComboDamage.Resolve(
    ResolveSourceId(context),
    contributingDice != null ? 0 : max, // ← el switch de camino
    contributingDice,
    _comboMultiplier);
```

Si no hay dados en absoluto (un behavior con `ComboValue` pero sin tirada), o si el máximo es ≤ 0 , el fallback devuelve 0 y `ApplyEffect` corta temprano sin invocar el pipeline.

NOTA HISTÓRICA — BUG-040

Este fallback es distinto del combo *Higher Number*, que sí es un combo real del catálogo con base 5 y prioridad 5. El fallback aplica cuando **ningún** combo matcheó, incluido Higher Number (que, siendo el combo comodín, matchea con cualquier selección no vacía — así que en la práctica el fallback solo corre si el catálogo de la clase no lo incluye).

9 El pipeline de aplicación del daño

El número que devuelve la fórmula es el **input** del `DamagePipeline`, no el daño final. El pipeline es el punto único por donde pasa **todo** el daño del juego — jugador, enemigos, trampas, mimic.

DAMAGEPIPELINE.RESOLVE — ETAPAS

0. Guard si `BaseDamage <= 0` → `Final = 0`, `weakness = 1`, corte
1. Outgoing NO IMPLEMENTADO — placeholder de `TECHNICAL §12.2`
emite `OnDamageOutgoing(src, tgt, damage)`
2. Weakness si `IsWeaknessHit` y `mult > 1`
 `damage = Mathf.RoundToInt(damage × mult)`
3. Incoming NO IMPLEMENTADO — placeholder
emite `OnDamageIncoming(src, tgt, damage)`
4. Escudo `absorbed = min(Shield(target), damage)`
 `damage -= absorbed`
 `Shield(target) = shieldBefore - absorbed`
emite `OnShieldChanged`
5. HP `newHp = max(0, hp - damage)`
 → clamp de HP mínimo (`IMinHpClampProvider`)
 → override de letalidad (`ILethalDamageOverride`)
 escribe `Health`, setea `WasLethal`
6. Evento `TypedEvent<DamageResolvedPayload>.Raise(...)`

9.1 Etapa 2 — weakness

La debilidad se activa cuando se cumplen **todas** estas condiciones:

1. `DamageContext.IsWeaknessHit` es `true` — lo setea `EffDealDamage` como `context.ComboResult?.IsMatch ?? false`. **Sin combo no hay debilidad posible.**
2. Hay un `IWeaknessChecker` registrado.
3. El target tiene entrada en el `IWeaknessRegistry`, con `comboId` no vacío.
4. Ese `comboId` es **exactamente igual** al del golpe.

El multiplicador es `data.mult` si es > 0 , o `RulesetS0.Weakness.DefaultMultiplier` si el registry trae 0. Si el resultado es ≤ 1.0 se normaliza a 1.0 y no se aplica nada.

`GetMultiplier` dispara `OnWeaknessHit` como side-effect (informativo, no transaccional). `PeekMultiplier` hace el mismo lookup **sin** disparar el evento — es lo que usa el preview.

"NONE" = SIN DEBILIDAD

Un enemigo con `WeaknessComboId` vacío resuelve a $\times 1.0$ en el lookup, no falla ni tira warning.

9.2 Etapa 4 — absorción de escudo

```
static int ComputeShieldAbsorbed(int shieldValue, int damage)
{
    if (shieldValue <= 0 || damage <= 0) return 0;
    return Mathf.Min(shieldValue, damage);
}
```

Absorción lineal 1:1, sin porcentajes ni reducción. Esta función es el **único** source of truth: la comparten `Resolve` (que además escribe y emite) y `Preview` (que solo computa), para que preview y golpe real no puedan driftear.

Dos flags derivados viajan en el payload para la UI:

- `BlockedByShield` = el daño residual quedó en 0 y hubo absorción > 0 .
- `ShieldBroken` = el escudo estaba arriba y quedó exactamente en 0 tras absorber.

9.3 Etapa 5 — los dos clamps de HP

Ambos se aplican en cadena, ambos **reescriben** `ctx.FinalDamage` para que el número flotante refleje el daño realmente infligido, no el calculado.

CLAMP	INTERFAZ	REGLA
HP mínimo Mimic — Feature#0046	<code>IMinHpClampProvider</code>	Piso por par (<i>target, source</i>): el daño no-jugador nunca baja al target de <code>minHp</code> . Clampeado a la vida actual, para no <i>curar</i> a un target que ya estaba por debajo del piso.
Override de letalidad tutorial	<code>ILethalDamageOverride</code>	Si el golpe sería letal, el target queda con <code>LethalOverrideRemainingHp = 10</code> y <code>WasLethal</code> queda en <code>false</code> — el DeathWatcher nunca lo ve. Los 10 HP son el 10% del pool base de 100, el mismo ratio que tenía 1 HP sobre el pool viejo de 10.

Si el target no tiene atributo `Health`, el pipeline loguea un warning y **descarta el daño** (`FinalDamage = 0`).

9.4 `Preview` — el gemelo sin side-effects

`DamagePipeline.Preview` corre **solo las etapas 2 y 4** — weakness (con `PeekMultiplier`) y absorción de escudo (computada, no escrita). No toca `Health`, no toca `Shield`, no emite eventos. Es lo que permite que el HUD muestre el daño post-mitigación real antes de confirmar.

9.5 Regla huérfana — "combo repetido = 0"

`IsRepeatOfPreviousCombo` sigue en el archivo pero **nadie la llama**. Vivía detrás de un pasivo global anti-repetición que se eliminó por dos razones: la presión ahora la ejerce el propio boss desde su árbol de IA

(`AINode_RotateBlock`), y anular daño sin aviso en pantalla se sentía como un bug. Se conservó el cálculo — no el wiring — por si se reintroduce con UI que lo comunique.

10 El escudo como recurso

10.1 Aplicación

`EffAddShield.ApplyEffect` es deliberadamente simple — sin pipeline, sin mitigación, sin cap:

```
int current    = attributes.GetAttribute<Shield>(targetGuid)?.Value ?? 0;
int newShield = current + amount;      // acumula sobre lo que ya había
attributes.SetAttributeValue<Shield, int>(targetGuid, newShield);
EventManager.Trigger(EventName.OnShieldChanged, targetGuid, newShield);
```

El stat `Shield` es un `BaseAttribute<int>` pelado: sin lógica propia, sin máximo, sin regeneración.

10.2 Consumo

Solo se consume en la etapa 4 del pipeline, cuando el dueño recibe daño. Absorción 1:1 hasta agotarse (§9.2).

10.3 Reset por turno

`ShieldResetHandler` escucha `OnTurnStarted` y pone el escudo del **dueño del turno que arranca** en 0:

```
void OnTurnStarted(params object[] args)
{
    var shieldAttr = _attributes.GetAttribute<Shield>(entityGuid);
    if (shieldAttr == null || shieldAttr.Value <= 0) return;
    _attributes.SetAttributeValue<Shield, int>(entityGuid, 0);
    EventManager.Trigger(EventName.OnShieldChanged, entityGuid, 0);
}
```

Este es el freno de balance que reemplazó al cap. El escudo que generarás en tu fase de defensa **dura hasta que te vuelva a tocar el turno**: cubre exactamente la ronda de enemigos intermedia.

CONSECUENCIA DE DISEÑO

Acumular escudo a través de turnos es imposible por construcción. Cualquier ítem o pasiva que quiera "escudo persistente" necesita otro mecanismo — este handler no distingue de dónde vino el escudo.

11 Daño de enemigos y otras fuentes

La fórmula $N \times M$ es exclusiva del ataque de combo del jugador (`DamageSource.ComboValue`). Todo lo demás produce su número por otra vía y entra al pipeline directo.

DAMAGESOURCE	CÓMO RESUELVE EL NÚMERO	QUIÉN LO USA
Constant	<code>_baseAmount</code> tal cual (0.999, autorado en el asset)	enemigos simples, trampas, daño fijo
FromReader	<code>Mathf.RoundToInt(_reader.Read(context) × _readerMultiplier)</code> El reader es polimórfico: lee stats de entidad en runtime.	enemigos que escalan con sus propios stats
ComboValue	Fórmula v3 completa (§1)	ataque de combo del jugador

Con `FromReader` y `_reader == null` el efecto devuelve 0 (no tira excepción). Con `ComboValue` y sin combo, cae al fallback de §8.

Los enemigos **sí** atraviesan el pipeline completo: su daño puede ser absorbido por el escudo del jugador y está sujeto a los clamps de HP. Lo que no tienen es debilidad — `IsWeaknessHit` requiere un `ComboResult.IsMatch`, que solo produce el jugador.

LA ESCALA × 10

El "daño enemigo × 10" del rebalanceo a vida 100 es un cambio **de datos, no de código**: los `_baseAmount` de los assets de enemigos se multiplicaron. No hay ningún factor global de escala en el pipeline.

12 Preview del HUD vs. golpe real

La regla de anti-drift es que **el preview llama a las mismas funciones que el golpe**. `DamageFormulaView` nunca re-deriva la fórmula.

ETAPA	GOLPE REAL	PREVIEW
Fórmula	<code>PlayerComboDamage.Resolve</code>	la misma función
Weakness	<code>GetMultiplier</code> (dispara evento)	<code>PeekMultiplier</code> (silencioso)
Escudo	<code>ComputeShieldAbsorbed</code> + write	la misma función, sin write
Redondeo con ítems	el ítem escribe al play scratch	<code>RoundNxM(bd.N + itemBonus, bd.M)</code>

12.1 El bono de ítems, en dorado

Los bonos *at-played* del inventario no están en el scratch durante el preview (se limpia al inicio del turno), así que se previsualizan aparte con `IInventoryService.GetComboDamageBonusPreview(comboId)` y se muestran en dorado (`#FFC93C`) separados del número base.

La suma es correcta según v3 — el bono entra a **N** y escala por **M**, no se suma post-fórmula como en v2:

```
int preWithItems = itemBonus != 0 && !bd.Blocked
    ? PlayerComboDamage.RoundNxM(bd.N + itemBonus, bd.M)
    : preMitigation;

// La mitigación se aplica por separado a base y total, así el "+ N" dorado
// refleja la contribución de los objetos YA mitigada.
int shownBase = Mitigate(preMitigation);
int shownTotal = Mitigate(preWithItems);
int itemPortion = shownTotal - shownBase;
```

12.2 Dependencia de orden

`Resolve` lee `LastComboScratch` de los servicios de pasivas y encantamientos, que se pueblan al procesar el mismo `ComboMatchedPayload`. El preview correcto depende de que esos servicios estén suscritos *antes* que la view. Es una dependencia de orden de suscripción, no de código, y está anotada en el archivo.

12.3 Fase de defensa

Cuando el chain entra en fase de defensa (`OnChainPhaseStarted` con índice > 0), la view cambia a modo escudo: llama a `PlayerComboShield.Resolve` con la base de `ResolvePlayerShieldBase(comboId)`. Sin `Mitigate` — el escudo no pasa por el pipeline.

LIMITACIÓN CONOCIDA

`GetComboDamageBonusPreview` solo suma `EffAddComboBonus`. Un ítem *at-played* **multiplicativo** no se previsualiza: el número mostrado será menor que el golpe real. Anotado como follow-up en `DamageFormulaView.cs:350-351`.

13 Ejemplos numéricos trabajados

Todos con el **Warrior vigente**: `BaseAttack = 5` , `BaseMaxHp = 100` , `BaseDamageTable` vacía (sin overrides por clase), habilidad con `_comboMultiplier = 1.0` .

#1 — Trío de 5, sin nada DAÑO

Dados holdeados `[5, 5, 5, 2, 1]` . Trío matchea con los índices 0, 1, 2.

comboBase (Trío, sin override)	22
+ <code>dmg_base_PJ (Attack.Value)</code>	5
+ <code>bonos_PJ</code>	0
+ Σ caras (5+5+5)	15
+ <code>bono_combo</code>	0
N	42
<code>M = 1.0 (scratch) × 1.0 (ability)</code>	1.0
Fórmula → <code>round(42 × 1.0)</code>	42
Pipeline: <code>weakness × 1.0</code> , sin escudo	42
Daño al HP	42

#2 — El mismo Trío, con debilidad y escudo enemigo DAÑO

Enemigo con `WeaknessComboId = combo.trio` , `mult = 1.5` , y 10 de escudo activo.

Fórmula (idéntica al #1)	42
× weakness 1.5 → <code>Mathf.RoundToInt(63.0)</code>	63
– escudo absorbido <code>min(10, 63)</code>	–10
Daño al HP	53

El escudo del enemigo queda en 0 → `ShieldBroken = true` en el payload. `BlockedByShield` es `false` porque quedó daño residual.

#3 — Higher Number: el doble conteo DAÑO

Dados [6, 3, 2, 2, 1]. Ningún combo real matchea; gana Higher Number con el 6.

comboBase = <code>_baseDamage (5) + cara más alta (6)</code>	11
+ <code>dmg_base_PJ</code>	5
+ Σ caras — <i>el mismo 6, otra vez</i>	6
N	22
Daño (M = 1.0)	22

Sin el doble conteo serían 16. La cara del dado pesa el doble en todos los combos de base dinámica (§5.3). Deuda de balance aceptada.

#4 — Escudo con Doble Par ESCUDO

Fase de defensa del chain. Dados [3, 3, 5, 5, 1], contribuyentes 0-3.

Gate: <code>GetShieldBase("combo.double_pair") = 2 > 0</code> ✓	—
comboBase (tabla de escudo , no de daño)	2
+ <code>dmg_base_PJ</code> (sí, el ATQ suma escudo)	5
+ Σ caras (3+3+5+5)	16
N	23
Escudo generado (M = 1.0)	23

Se suma al stat `Shield` del jugador. Absorberá los próximos 23 de daño y se reseteará a 0 cuando arranque el siguiente turno del jugador.

#5 — Higher Number en fase de escudo: el gate corta ESCUDO

Misma tirada del #3, pero en fase de defensa.

<code>GetShieldBase("combo.higher_number")</code>	0
Escudo generado — la fórmula ni corre	0

Sin el gate, esta tirada habría dado `0 + 5 + 6 = 11` de escudo con un combo que el diseño no quiere que defienda. El Warrior tiene la entrada explícitamente autorada en 0 para documentar la intención.

#6 — Con scratch: pasiva multiplicativa + encanto aditivo AMBOS

Par de 6 ([6, 6, 4, 2, 1]) con una pasiva de `ComboDamageMultiplier = 1.5` y un encantamiento de `BonusComboDamage = +4`.

comboBase (Par)	8
+ dmg_base_PJ	5
+ Σ caras (6+6)	12
+ bono_combo (encanto)	4
N	29
M = 1.5 (pasiva) $\times$ 1.0 (ability)	1.5
<code>round(29 $\times$ 1.5) = round(43.5)</code> AwayFromZero	44

En v2 el multiplicador solo habría escalado el término de combo: `8 $\times$ 1.5 + 5 + 12 + 4 = 33`. La v3 escala todo N — 11 puntos de diferencia sobre el mismo build.

#7 — Habilidad con perilla < 1 y bloqueo DAÑO

El Trío del #1 con una habilidad de "golpe rápido" (`_comboMultiplier = 0.75`).

N (idéntico al #1)	42
M = 1.0 $\times$ 0.75	0.75
<code>round(42 $\times$ 0.75) = round(31.5)</code> AwayFromZero	32

Si además alguna fuente hubiera levantado `BlockComboDamage`, el resultado sería **0** y el producto ni se calcularía — `Blocked = true` en el breakdown y el log mostraría "🚫 BLOQUEADO por scratch".

14 Datos vigentes del Warrior

Assets/Rollgeon/Classes/CH_Warrior.asset

14.1 Stats base

BaseMaxHp	100
BaseAttack	5
BaseSpeed	5

14.2 BaseDamageTable (override de daño por clase)

Vacía. Los 9 combos usan el `_baseDamage` global de su SO (tabla de §5.1).

14.3 ShieldBaseTable

Ocho entradas autoradas. `combo.brute_force` **no tiene entrada** → no genera escudo.

COMBO	COMBOID	SHIELDBASE	GENERA ESCUDO
Higher Number	<code>combo.higher_number</code>	0	no — entrada explícita en 0
Par	<code>combo.pair</code>	1	sí
Doble Par	<code>combo.double_pair</code>	2	sí
Trío	<code>combo.trio</code>	3	sí
Full House	<code>combo.full_house</code>	4	sí
Escalera	<code>combo.ladder</code>	5	sí
Póker	<code>combo.poker</code>	6	sí
Generala	<code>combo.generala</code>	8	sí
Fuerza Bruta	<code>combo.brute_force</code>	—	no — sin entrada

PESO RELATIVO

La `ShieldBaseTable` va de 1 a 8, mientras que los bases de daño van de 5 a 90. Contra un N típico de 20-45, el `shieldBase` aporta menos del 10%: **el escudo del Warrior está dominado por ATQ + Σcaras**, no por el combo elegido. Cambiar Par de 1 a 2 mueve el escudo un punto. Es el balance pendiente que ya estaba anotado como TBD.

14.4 Wiring del ataque base

El behavior "Base Attack" es un `EffChain` de dos fases:

FASE	EFEECTO RELEVANTE	CONFIG
Attack	<code>EffDealDamage</code> dentro de un <code>EffPlaySequence</code> , disparado por el evento <code>hit</code> de la animación	<code>_damageSource =</code> <code>ComboValue</code> <code>_comboMultiplier = 1.0</code> <code>_attackKind =</code> <code>BasicAttack</code>
Shield	<code>EffAddShield</code> + <code>EffPlayFeedback("shield.basic")</code>	<code>_shieldSource =</code> <code>ComboValue</code> <code>_comboMultiplier = 1.0</code> <code>OverrideBoardType =</code> <code>true</code>

Que el `EffDealDamage` viva dentro del `EffPlaySequence` en el evento `hit` es exactamente la razón por la que el canal at-played necesita `LastPlayScratch` persistente (§3.2).

15 Gotchas, deudas y hallazgos

15.1 Redondeo inconsistente entre fórmula y pipeline BUG MENOR

La fórmula usa `MidpointRounding.AwayFromZero` explícitamente, con una función dedicada y un comentario que dice que `Mathf.RoundToInt` haría lo incorrecto. La etapa de weakness del pipeline usa `Mathf.RoundToInt`.

CASO CONCRETO DONDE DIFIEREN

daño de la fórmula = 43, weakness = 1.5

$43 \times 1.5 = 64.5$

`Mathf.RoundToInt(64.5f)` → **64** (banker's: al par más cercano)

`RoundNxM(43, 1.5f)` → **65** (AwayFromZero, el criterio canónico)

Aparece en `DamagePipeline.cs:88` (Resolve) y `:221` (Preview). Como **ambos** usan el mismo redondeo, preview y golpe real coinciden: no hay drift visible para el jugador. Es 1 punto de daño en casos de midpoint, y contradice la intención documentada de `RoundNxM`.

15.2 Doble conteo en combos de base dinámica DEUDA ACEPTADA

Higher Number, Suma X y Fuerza Bruta meten la cara del dado en `comboBase` y en Σ caras (§5.3, ejemplo #3). Documentado y aceptado hasta el próximo pase de balance. El efecto es más pronunciado en Fuerza Bruta, donde se duplican **los cinco** dados.

15.3 Fuerza Bruta: dos campos de base TRAMPA DE AUTORÍA

CAMPO	VALOR	PARA QUÉ SE USA
<code>_baseDamage</code> (heredado)	30	solo <code>Priority</code> — el orden de selección de combos
<code>_baseDamageConfigurable</code>	5	el daño real — lo que usa el <code>Detect</code> override

Fuerza Bruta compite en prioridad como si pegara 30 (por encima de Trío, por debajo de Full House) pero su piso plano es 5. Es deliberado — el rol de diseño es "combo consuelo" — pero un designer que edite `_baseDamage` esperando cambiar el daño va a mover la prioridad sin tocar el daño. Lo mismo aplica al `flatBaseOverride` de la tabla por clase: reemplaza `_baseDamageConfigurable`, no `_baseDamage`.

15.4 Tooltips desactualizados a v2 DOCUMENTACIÓN

Tres textos siguen describiendo la fórmula v2, incluyendo uno visible al jugador:

UBICACIÓN	TEXTO ACTUAL	POR QUÉ ESTÁ MAL
<code>EffAddShield.cs:38-40</code> [Tooltip] de inspector	"escala solo el término base del combo... ATQ y bonos planos no se multiplican"	En v3 todo N se multiplica por M.
<code>EffAddShield.BuildTooltip</code> visible al jugador	"Escudo: ATQ (x) + base del combo × multi de dados"	El multiplicador por tipo de dado no existe desde v3.
<code>EffDealDamage.BuildTooltip</code> visible al jugador	"Daño: ATQ + puntaje del combo"	Omite Σcaras, los tres canales de scratch y la perilla de habilidad.

15.5 Etapas del pipeline sin implementar INCOMPLETO

Las etapas 1 (outgoing) y 3 (incoming) de TECHNICAL §12.2 son comentarios. Hoy **no existe ninguna forma de escalar daño entrante o saliente por stat** — los stats `OutgoingDamageMultiplier` e `IncomingDamageMultiplier` no están definidos. Cualquier diseño que asuma "reducción de daño recibido del 20%" no tiene dónde engancharse; el único mitigador que existe es el escudo.

15.6 Ítems multiplicativos sin preview LIMITACIÓN

`GetComboDamageBonusPreview` solo agrega `EffAddComboBonus`. Un ítem at-played con `ComboDamageMultiplier` se aplicará en el golpe real pero no aparecerá en el número previsualizado (§12.1).

15.7 Código huérfano vivo

- `DamagePipeline.IsRepeatOfPreviousCombo` — conservado a propósito, sin call sites (§9.5).
- `Combo_SumaX` — clase completa y testeada, sin ningún `.asset` autorado.
- `BaseComboSO.ComputeCount` / `ValueMultipliers` / `GeneralMultiplier` — la fórmula §5.1.1 del TECHNICAL. Todos los assets tienen `ValueMultipliers` en ceros y `GeneralMultiplier = 1` ; **la v3 no la consume**.

A Apéndice — mapa de archivos

ARCHIVO	ROL
Combat/Damage/PlayerComboDamage.cs	La fórmula. Fuente de verdad única de N×M y de RoundNxM .
Combat/Damage/PlayerComboShield.cs	Gate de base 0 + delegación a la fórmula con Kind = Shield .
Combat/Damage/DamageBreakdown.cs	Struct de salida con cada término. Invariante Final == RoundNxM(N, M) .
Combat/Damage/ContributingDie.cs	Tripla slot + cara + tipo.
Combat/Damage/ContributingDiceResolver.cs	Mapeo índices del subset → slots del bag → caras y tipos.
Combat/Damage/DamageDebugLogger.cs	Log de composición y aplicación. [Conditional] : overhead cero en release.
Combat/Pipelines/DamagePipeline.cs	Aplicación: weakness, escudo, HP, clamps, evento. Resolve + Preview .
Combat/Weakness/WeaknessChecker.cs	Lookup del multiplicador de debilidad. GetMultiplier vs PeekMultiplier .
Combat/ShieldResetHandler.cs	Reset del escudo en OnTurnStarted .
Effects/Concretes/EffDealDamage.cs	Punto de entrada del daño. Resuelve fuente, targets, fallback sin combo.
Effects/Concretes/EffAddShield.cs	Punto de entrada del escudo. Segundo gate (sheet + combold).
Effects/Concretes/EffMultiplyComboDamage.cs	Escribe ComboDamageMultiplier al scratch (*=).
Heroes/ContractSheet.cs	BaseDamageTable , ShieldBaseTable y sus accessors.
Combos/BaseComboSO.cs	Base de combos: _baseDamage , Priority , Detect .
Combos/Concretes/Combo_*.cs	Reglas por combo; tres overridean Detect con base dinámica.
Upgrades/Dice/EnchantmentScratch.cs	Struct del scratch compartida por los tres canales.
UI/HUD/DamageFormulaView.cs	Preview. Llama a las mismas funciones que el golpe.
UI/HUD/Breakdown/DamageBreakdownView.cs	Render animado del N×M.

Cobertura de tests

Assets/Scripts/Rollgeon/Combat/Tests/

- PlayerComboDamageTests — términos de N, composición de M, bloqueo, redondeo.
- PlayerComboShieldTests — gate de base 0, paridad con la fórmula de daño.
- DamageBreakdownTests — invariante Final == RoundNxM(N, M) .
- DamagePipelineTests , DamagePipelineLethalOverrideTests , DamagePipelineMinHpClampTests .
- ContractSheetBaseDamageTests , ContractSheetShieldBaseTests — lookup de tablas.

- `ContributingDiceResolverTests` — mapeo de índices.

B Apéndice — el log de diagnóstico

`DamageDebugLogger` emite dos bloques por golpe, en la consola de Unity, con la **misma struct** que produjo la fórmula — nunca una copia paralela. Está marcado con `[Conditional("UNITY_EDITOR")]` y `[Conditional("DEVELOPMENT_BUILD")]`: las llamadas se eliminan del IL en release. El toggle `DamageDebugLogger.Enabled` permite apagarlo en runtime sin recompilar.

Bloque 1 — composición

```
■■■ DMG · COMPOSICIÓN - daño base del PLAYER ■■■ src=a3f91c
  combo_base: 22
+ dmg_base_PJ (ATQ.Value): 5
+ bonos_PJ (ATQ.Modified - Value): 0
+ Σ caras contribuyentes: 15
+ bono_combo (passives + enchants + items): 0
N = 42
M = 1 (scratch) × 1 (ability) = 1
= TOTAL = N × M = 42 × 1 = 42 → entra al DamagePipeline como BaseDamage
```

En modo escudo la banda cambia a `SHIELD · COMPOSICIÓN` en cyan y la última línea dice "→ se suma al atributo Shield (EffAddShield)". Si hubo bloqueo, todo el bloque se reemplaza por `🚫 BLOQUEADO por scratch (BlockComboDamage)`.

Bloque 2 — aplicación

```
■■■ DMG · APLICACIÓN - DamagePipeline ■■■ BasicAttack src=a3f91c → tgt=7b2e04
BaseDamage (entrante): 42
× weakness = 1.5 (¡debilidad!)
escudo: 7b2e04 tenía 10, absorbe 10
HP: 100 → 47
= FinalDamage = 53
```

Este bloque cubre **todo** el daño del juego, no solo el del jugador — el `AttackKind` del encabezado permite distinguir la fuente. Si el golpe fue letal, la última línea agrega `🚫 LETAL`.

SOBRE EL FORMATO

La consola IMGUI de Unity no soporta color de fondo real (no hay etiqueta `<mark>`). Las bandas se emulan con bloques `■` coloreados sobre `<color>` y `<b>` — lo más cercano a un highlight que la consola permite.